

# QUANT: POST-QUANTUM CRYPTOGRAPHY PLATFORM

The Complete Technical Handbook: Quantum Threat Model, NIST PQC Standards, End-to-End Encryption Mechanics, Cyber Attack Defense, and Unique Innovations

Platform Version: v3.0 | Standards Compliance: NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), RFC 5869 (HKDF), NIST SP 800-232 (Ascon) | Author: QUANT Engineering Team

## 1. Quantum Computing Fundamentals & The Cryptographic Threat Model

To understand why the **QUANT Platform** was built, one must first understand the fundamental shift from classical computing to quantum computing, and how it completely dismantles the mathematical foundations of modern internet security.

### 1.1 Classical Bits vs. Quantum Qubits

- **Classical Computers:** Store and process information in binary **bits** (either a 0 or a 1). Every calculation happens sequentially or in parallel threads across physical silicon transistors.
- **Quantum Computers:** Exploit the laws of quantum mechanics using **qubits**. Due to *Superposition*, a qubit can exist as a linear combination of  $|\alpha\rangle + |\beta\rangle$  simultaneously. Due to *Entanglement*,  $N$  qubits can simultaneously represent  $2^N$  states, allowing quantum algorithms to explore vast mathematical solution spaces exponentially faster than any classical supercomputer.

### 1.2 How Quantum Computing Destroys Classical Cryptography

Virtually all modern secure protocols (HTTPS/TLS, RSA, SSH, ECC, Diffie-Hellman, Bitcoin, Apple iMessage, WhatsApp) rely on two specific mathematical problems that are hard for classical computers:

- **Integer Factorization Problem (IFP):** Given a large number  $N = p \times q$  (e.g., RSA-2048), finding the prime factors  $p$  and  $q$  takes billions of years on classical computers.
- **Discrete Logarithm Problem (DLP / ECDLP):** Given  $g$  and  $g^x \pmod p$  or elliptic curve point  $P = k \cdot G$ , finding the private scalar  $x$  or  $k$  is computationally infeasible classically.

In 1994, mathematician Peter Shor published **Shor's Algorithm**. On a Cryptographically Relevant Quantum Computer (CRQC), Shor's algorithm solves both Integer Factorization and Discrete Logarithms in **polynomial time**  $\mathcal{O}((\log N)^3)$ . An RSA-2048 key that requires  $2^{112}$  operations (~billions of years) on classical supercomputers can be factored in approximately **10 to 30 seconds** on a CRQC with ~4,096 logical qubits.

### 1.3 Shor's Algorithm vs. Grover's Algorithm

| Algorithm / Cipher                                            | Classical Security                     | Quantum Attack & Impact                                                                                           | Quantum Posture / Status                                            |
|---------------------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| RSA-2048 / RSA-4096<br>(Asymmetric / PKI)                     | 112 / 128 bits<br>(Secure classically) | <b>Shor's Algorithm:</b> Polynomial time reduction. Private keys completely recovered in seconds.                 | <b>COMPLETELY BROKEN</b><br>(Must be replaced)                      |
| ECDH / ECDSA (Curve25519, P-256)<br>(Asymmetric / Signatures) | 128 bits<br>(Secure classically)       | <b>Shor's Algorithm:</b> Solves ECDLP in $\mathcal{O}(n^3)$ . Reconstructs private signing and exchange keys.     | <b>COMPLETELY BROKEN</b><br>(Must be replaced)                      |
| AES-128 Symmetric Cipher                                      | 128 bits                               | <b>Grover's Algorithm:</b> Quadratic speedup $\mathcal{O}(\sqrt{N})$ . Effective security reduced to 64 bits.     | <b>POTENTIALLY VULNERABLE</b><br>(Brute-forceable by nation-states) |
| AES-256 / ChaCha20-Poly1305                                   | 256 bits                               | <b>Grover's Algorithm:</b> Effective security reduced from 256 to 128 bits ( $\$3.4 \times 10^{38}$ quantum ops). | <b>QUANTUM SECURE</b><br>(Immune to brute force)                    |
| SHA3-256 / SHA3-512<br>(Cryptographic Hashes)                 | 256 / 512 bits                         | <b>Brassard-Høyer-Tapp (BHT):</b> Collision search takes $\mathcal{O}(N^{1/3})$ . SHA3-512 retains 170+ bits.     | <b>QUANTUM SECURE</b><br>(Zero practical collision risk)            |

1.4 "Harvest Now, Decrypt Later" (HNDL) - Why We Must Act Now

Many people ask: "If quantum computers are still years away, why do we need QUANT today?" The answer is **HNDL Espionage**: Foreign intelligence agencies and cybercriminals are actively capturing and archiving encrypted emails, chat communications, military secrets, and financial transactions over internet cables today. Even though they cannot decrypt it today, the moment a quantum computer is activated (known as **Q-Day**), they will retroactively decrypt the entire historical archive. Any data with a secrecy lifespan over 5 to 10 years is already compromised if protected only by RSA or ECC. QUANT solves this immediately.

2. What is Post-Quantum Cryptography (PQC)?

**Post-Quantum Cryptography (PQC)** refers to cryptographic algorithms engineered to run on existing, classical computers and microchips today, but whose mathematical security relies on problems so complex that even quantum computers cannot solve them.

2.1 Module Learning with Errors (M-LWE) Lattice Mathematics

Instead of prime numbers, PQC algorithms like **ML-KEM** and **ML-DSA** are built on **High-Dimensional Lattice Grid Geometry**. In Module-LWE, an attacker is given a matrix equation with added random Gaussian noise:  $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$ . Finding the secret vector  $\mathbf{s}$  requires finding the shortest vector in an  $n$ -dimensional lattice (the Shortest Vector Problem, or SVP). Shor's quantum algorithm cannot exploit this structure, making lattice cryptography completely quantum-proof.

2.2 Official NIST PQC Standards (Standardized August 2024)

| Standard & Algorithm                                              | Type & Category                        | Mathematical Foundation                                             | Role in QUANT Platform                                                                                                         |
|-------------------------------------------------------------------|----------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>ML-KEM</b><br>(NIST FIPS 203)<br><i>Kyber-512 / 768 / 1024</i> | Key Encapsulation Mechanism (KEM)      | Module Learning With Errors (M-LWE) over polynomial rings.          | Securely establishes symmetric session keys between users during chat and mail handshakes without transmitting raw keys.       |
| <b>ML-DSA</b><br>(NIST FIPS 204)<br><i>Dilithium-44 / 65 / 87</i> | Digital Signature Algorithm            | Module Short Integer Solution (M-SIS) with Fiat-Shamir with Aborts. | Digitally signs user identity, handshake tokens, and message packets. Prevents Man-in-the-Middle (MITM) forgery.               |
| <b>SLH-DSA</b><br>(NIST FIPS 205)<br><i>SPHINCS+</i>              | Stateless Hash-Based Digital Signature | W-OTS+ one-time signatures and Merkle authentication trees.         | Fallback ultra-conservative signature scheme. Relies strictly on SHA-2 / SHAKE hash security; immune to lattice breakthroughs. |
| <b>Ascon-128a</b><br>(NIST SP 800-232)                            | Lightweight AEAD Symmetric Cipher      | Permutation-based sponge duplex construction.                       | Optimized authenticated encryption engine for resource-constrained IoT nodes and edge microcontrollers.                        |

3. QUANT Platform Architecture & Technology Blueprint

The QUANT platform is designed as an asynchronous, highly concurrent web application utilizing Flask, Eventlet, Flask-SocketIO, and native C-compiled post-quantum libraries (`liboqs`).

**SYSTEM ARCHITECTURE TOPOLOGY & SUBSYSTEMS**

[ Client / Browser UI ] • Cyber Glassmorphism • Three.js 3D WebGL Entangled Singularity • Web Audio API Synthesizer • Chart.js Telemetry

■ (HTTP REST APIs & Bidirectional WebSockets - WSS)

[ Application Server ] • Flask 3.x WSGI • Eventlet Async Green-Threads • Flask-SocketIO Message Broker • Bcrypt Auth

■

[ Cryptographic Engine ] • liboqs Native C (ML-KEM, ML-DSA, SLH-DSA) • cryptography (AES-GCM, X25519, HKDF-SHA384, SHA3-512)

■

[ Persistence Layer ] • SQLite / SQLAlchemy ORM • Ciphertext-Only Messages • Encrypted Email & Vault Files • Security Audit Logs

### 3.1 Key Modules in the Codebase

- `app/crypto/`: Houses all classical, hybrid, PQC (liboqs), and symmetric algorithms with dynamic Windows/Linux DLL loaders.
- `app/chat/events.py`: Manages real-time WebSockets, 2-party interactive handshake negotiation, and monotonic sequence anti-replay tracking.
- `app/mail/mail_routes.py`: Encrypted asynchronous email system with context-bound HKDF key derivation and ML-DSA signatures.
- `app/files/file_routes.py`: Zero-knowledge encrypted file vault with per-file AES-256-GCM and SHA3-512 / SHAKE-256 digests.
- `app/api/routes.py`: Implements the Live Attack Simulation Lab and the Microsecond Benchmarking / Quantum Effort engine.
- `app/keys/key_routes.py`: Manages identity key generation, public key directories, and instant key rotation with session revocation.

## 4. Deep-Dive: How Encryption & Decryption Works Step-by-Step

Here is the exact mathematical and operational journey of a message sent through the QUANT Platform:

### Phase 1: Hybrid Ephemeral Key Generation

When User A selects User B and initiates a session, Alice generates an ephemeral keypair:

1. **Classical**: Ephemeral X25519 Private Key ( $\$sk_{\{A\}}$ ) and Public Key ( $\$pk_{\{A\}}$ ).
  2. **Post-Quantum**: Ephemeral ML-KEM-768 Secret Key ( $\$sk_{\{KEM,A\}}$ ) and Public Key ( $\$pk_{\{KEM,A\}}$ ).
- Alice sends an `initiate_handshake` packet with her public keys to Bob via WebSocket.

### Phase 2: Bob's Interactive Consent & Encapsulation

Bob receives an interactive UI modal showing Alice's cryptographic identity. When Bob clicks "Accept & Connect":

1. Bob generates his ephemeral X25519 keypair ( $\$sk_{\{B\}}$ ,  $pk_{\{B\}}$ ) and computes the Diffie-Hellman secret:  $\$K_{\{\text{classical}\}} = \text{\texttt{X25519}}(sk_{\{B\}}, pk_{\{A\}})$ .
  2. Bob encapsulates a quantum secret against Alice's ML-KEM public key:  $\$(c_{\{\text{kem}\}}, K_{\{\text{pqc}\}}) = \text{\texttt{ML-KEM-Encapsulate}}(pk_{\{KEM,A\}})$ .
  3. **Identity Authentication**: Bob signs  $\$(pk_{\{B\}} \parallel c_{\{\text{kem}\}})$  with his long-term **ML-DSA-65 Private Key** ( $\$sk_{\{\text{mldsa},B\}}$ ).
- Bob transmits  $\$(pk_{\{B\}}, c_{\{\text{kem}\}}, \text{\texttt{Signature}})$  back to Alice.

### Phase 3: Hybrid Key Derivation via HKDF-SHA-384

1. Alice receives Bob's payload and verifies Bob's ML-DSA-65 digital signature using Bob's verified public key. (If signature fails, the handshake is instantly aborted!)
2. Alice computes  $\$K_{\{\text{classical}\}} = \text{\texttt{X25519}}(sk_{\{A\}}, pk_{\{B\}})$  and decapsulates  $\$K_{\{\text{pqc}\}} = \text{\texttt{ML-KEM-Decapsulate}}(c_{\{\text{kem}\}}, sk_{\{KEM,A\}})$ .
3. Both parties now possess identical secrets ( $\$K_{\{\text{classical}\}}$  and  $\$K_{\{\text{pqc}\}}$ ). They feed both into HKDF-SHA-384:  

$$\$K_{\{\text{session}\}} = \text{\texttt{HKDF-SHA-384}}(K_{\{\text{classical}\}} \parallel K_{\{\text{pqc}\}}, \text{\texttt{salt}}=\text{\texttt{None}}, \text{\texttt{info}}=\text{\texttt{b"pqc-secure-platform-hybrid-v1"}}, \text{\texttt{length}}=32)$$
4. **The Result**: A 256-bit symmetric key that is dual-protected: unbreakable by classical computers (due to X25519) and unbreakable by quantum computers (due to ML-KEM).

### Phase 4: Message Encryption & Context-Bound Transport

When Alice types plaintext and clicks "Encrypt & Send":

1. **Monotonic Sequence Number**: Alice increments local counter  $\$Seq = Seq + 1$ .
2. **Random IV**: Generates a cryptographically secure 96-bit random Initial Vector ( $\$IV$ ) via `os.urandom(12)`.
3. **Authenticated Data Binding (AAD)**: Sets  $\text{\texttt{AAD}} = \text{\texttt{"hybrid-"} \parallel Seq}$ . This cryptographically locks the message to its sequence number.
4. **AES-256-GCM Encryption**: Computes  $\$(Ciphertext, Tag) = \text{\texttt{AES-GCM-Encrypt}}(K_{\{\text{session}\}}, IV, Plaintext, \text{\texttt{AAD}})$ .
5. **Digital Signature**: Alice signs the raw ciphertext bytes with her private ML-DSA-65 key.
6. The complete packet is sent over WebSockets. **Plaintext never touches the server or database!**

Phase 5: 3-Tier Security Validation & Decryption

- Upon receiving the packet, Bob's client performs 3 sequential security checks:
- **Tier 1 (Anti-Replay Filter):** Checks if \$Seq\$ was already seen. If duplicate, packet is dropped immediately.
  - **Tier 2 (ML-DSA Signature Verification):** Verifies Alice's post-quantum signature against the ciphertext.
  - **Tier 3 (AES-GCM 128-bit Tag Check):** AES-256-GCM verifies the Galois polynomial tag over Ciphertext + AAD. If even a single bit was modified in transit, an InvalidTag exception is raised and data is discarded.

5. Live Cyber Attack Defense & Security Matrix

QUANT includes a dedicated cyber laboratory demonstrating real-world attack vectors blocked by its cryptographic engine:

| Attack Vector              | Simulated Adversary Action                                                       | QUANT Mathematical Defense                                            | Defense Outcome                                                     |
|----------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------|
| Shor's Quantum Attack      | Adversary executes Shor's algorithm on a quantum computer to factor public keys. | Module-LWE Lattice grid geometry (NIST FIPS 203 ML-KEM-768).          | <b>BLOCKED:</b> Immune to Shor's algorithm ( $>10^{30}$ years).     |
| Man-in-the-Middle (MITM)   | Attacker intercepts KEM exchange and injects rogue public keys.                  | Handshake payloads are signed with ML-DSA-65 / SLH-DSA identity keys. | <b>BLOCKED:</b> Signature verification fails; handshake terminates. |
| Ciphertext Bit Tampering   | Adversary flips bits in transit (e.g. changing "Pay \$10" to "Pay \$90").        | AES-256-GCM GHASH polynomial authentication tag verification.         | <b>BLOCKED:</b> InvalidTag exception raised; packet purged.         |
| Packet Replay Attack       | Attacker captures valid encrypted packet and retransmits it repeatedly.          | Monotonic sequence tracker and AAD context binding in AEAD.           | <b>BLOCKED:</b> Duplicate sequence dropped; ATTACK_REPLAY alert.    |
| Unauthorized Key Injection | Attacker attempts to decrypt ciphertext using an unauthorized key.               | Galois Counter Mode cryptographic tag mismatch.                       | <b>BLOCKED:</b> Decryption fails cleanly without leaking plaintext. |

6. Project Uniqueness & Competitive Advantage

Why does the QUANT Platform stand out in the global cybersecurity landscape?

| Platform             | Key Exchange                              | Digital Signatures                   | Attack Simulation Lab                   | Post-Quantum Rating                               |
|----------------------|-------------------------------------------|--------------------------------------|-----------------------------------------|---------------------------------------------------|
| Our QUANT Platform   | Hybrid X25519 + ML-KEM (512 / 768 / 1024) | ML-DSA-65 & SLH-DSA (SPHINCS+)       | YES (6 Live Attack Vectors + Telemetry) | <b>100% QUANTUM-PROOF</b> (Full KEM + Signatures) |
| Signal (PQXDH)       | Hybrid X25519 + ML-KEM-768                | Ed25519 (Classical - Broken by Shor) | No (Closed production app)              | <b>PARTIAL</b> (Vulnerable signatures)            |
| Apple iMessage (PQ3) | Kyber-768 + P-256 Ratchet                 | ECDSA (Classical - Broken by Shor)   | No (Proprietary Apple ecosystem)        | <b>PARTIAL</b> (Vulnerable signatures)            |
| WhatsApp             | Curve25519 (Classical)                    | Ed25519 (Classical)                  | No                                      | <b>VULNERABLE</b> (Broken on Q-Day)               |
| Telegram             | RSA-2048 / Diffie-Hellman                 | RSA (Classical)                      | No                                      | <b>CRITICAL RISK</b> (Broken on Q-Day)            |

6.1 The 5 Key Pillars of QUANT Uniqueness:

- 1. **True End-to-End Post-Quantum Security:** Unlike Signal or Apple PQ3 (which only upgraded key exchange but still use classical signatures vulnerable to quantum MITM attacks), QUANT implements **both Post-Quantum KEM and Post-Quantum Signatures (ML-DSA & SLH-DSA)**.
- 2. **Live Interactive Cyber Warfare Lab:** Built-in real-time demonstration engine that proves mathematical defense against 6 attack vectors with live packet inspector.
- 3. **Multi-NIST Algorithm Agility:** Users can dynamically switch between NIST Level 1 (512), Level 3 (768), Level 5 (1024), Stateless Hash SPHINCS+, and IoT Ascon-128a.
- 4. **Zero-Knowledge Encrypted Vault:** Files encrypted with per-file keys wrapped in master keys, authenticated by SHA3-512 quantum-proof digests.
- 5. **Three.js Hardware-Accelerated 3D Quantum Visualization:** Stunning WebGL visualizer depicting entangled photon dynamics and quantum tunneling with Web Audio API sound synthesis.

7. Teammate Quick-Start & Development Workflow

To start the platform and run verification tests on any operating system:

**QUICK START COMMANDS (POWERSHELL / BASH)**

- 1. Install Dependencies: `python -m pip install -r requirements.txt` (or run `INSTALL_DEPENDENCIES.bat`)
- 2. Launch Web Application: `python run.py` (or run `START_SERVER.bat`)
- 3. Access Portal: Open browser to `http://127.0.0.1:5000`
- 4. Run Complete Automated Test Suite: `python -m pytest -q` (Runs all crypto, attack, and integration tests)